PostgreSQL Production Performance Troubleshooting

Top 5 Critical Performance Issues
Detection, Diagnosis & Resolution Guide

Applicable Versions: PostgreSQL 15, 16, 17

Environment: Private Cloud VMs — Single Node & Streaming Replication

Table of Contents

1. Executive Summary
   - 1.1 Environment Overview
   - 1.2 Top 5 Issues at a Glance
2. Issue 1: Bloated Tables & Missing VACUUM / AUTOVACUUM Tuning
   - 2.1 What Is Table Bloat?
   - 2.2 Root Causes
   - 2.3 Detection & Diagnosis
   - 2.4 Resolution & Remediation
   - 2.5 Monitoring Recommendations
3. Issue 2: Missing or Inefficient Indexes & Poor Query Plans
   - 3.1 What Is the Problem?
   - 3.2 Root Causes
   - 3.3 Detection & Diagnosis
   - 3.4 Resolution & Remediation
   - 3.5 Monitoring Recommendations
4. Issue 3: Lock Contention & Long-Running Transactions
   - 4.1 What Is the Problem?
   - 4.2 Root Causes
   - 4.3 Detection & Diagnosis
   - 4.4 Resolution & Remediation
5. Issue 4: Suboptimal Memory & Configuration Settings
   - 5.1 What Is the Problem?
   - 5.2 Root Causes
   - 5.3 Detection & Diagnosis
   - 5.4 Resolution & Remediation
6. Issue 5: Streaming Replication Lag & Replication Slot Bloat
   - 6.1 What Is the Problem?
   - 6.2 Root Causes
   - 6.3 Detection & Diagnosis
   - 6.4 Resolution & Remediation

7. Appendix: Quick Reference & Diagnostic Toolkit

7.1 Essential Diagnostic Extensions

7.2 Daily Health Check Script

7.3 Recommended Monitoring Stack

1. Executive Summary

This document provides an in-depth analysis of the five most critical performance issues encountered in PostgreSQL production environments running on private cloud virtual machines. Each issue includes a root cause analysis, detection methods, step-by-step troubleshooting procedures, and recommended preventive measures.

The guidance is tailored for PostgreSQL versions 15, 16, and 17, covering both single-node deployments and dual-node primary/secondary configurations using streaming replication with replication slots.

> **Target Audience**
> PostgreSQL Database Administrators, DevOps Engineers, and SREs managing production PostgreSQL clusters on private cloud infrastructure.

1.1 Environment Overview

Component	Details
PostgreSQL Versions	15.x, 16.x, 17.x
Infrastructure	Private Cloud VMs (company-managed)
Topology	Single-node standalone + Dual-node (Primary/Secondary)
Replication	Streaming replication with replication slots
Operating System	Linux (RHEL/Ubuntu-based)

1.2 Top 5 Issues at a Glance

#	Issue	Severity	Impact Area
1	Bloated Tables & Missing VACUUM/AUTOVACUUM	Critical	Storage, Query Speed, XID Wraparound
2	Missing or Inefficient Indexes & Poor Query Plans	Critical	Query Latency, CPU, I/O
3	Lock Contention & Long-Running Transactions	High	Concurrency, Throughput, Replication Lag

#	Issue	Severity	Impact Area
4	Suboptimal Memory & Configuration Settings	High	Buffer Cache Hit Ratio, Sorting, I/O
5	Streaming Replication Lag & Slot Bloat	High	Data Durability, Failover, Storage

2. Issue 1: Bloated Tables & Missing VACUUM / AUTOVACUUM Tuning

2.1 What Is Table Bloat?

PostgreSQL uses Multi-Version Concurrency Control (MVCC) to allow concurrent reads and writes without locking. When a row is updated or deleted, the old version (a "dead tuple") is not immediately removed. It remains on disk until a VACUUM operation reclaims the space. When VACUUM does not run frequently enough, dead tuples accumulate, causing **table bloat**.

Table bloat leads to larger table sizes on disk, slower sequential scans (PostgreSQL must read through dead tuples), degraded index performance, and in the worst case, **Transaction ID (XID) wraparound** — a catastrophic event where PostgreSQL shuts down to prevent data corruption.

2.2 Root Causes

- Autovacuum is disabled or has overly conservative settings (high thresholds, long nap times).
- Very large tables where autovacuum cannot finish before the next cycle of dead tuples accumulates.
- Long-running transactions that hold back the visibility horizon, preventing VACUUM from reclaiming dead tuples.
- High UPDATE/DELETE workloads without corresponding VACUUM frequency tuning.
- Manual VACUUM operations cancelled or interrupted by maintenance windows.
- On replication setups: *hot_standby_feedback = on* with long-running queries on the standby holding back VACUUM on the primary.

2.3 Detection & Diagnosis

Step 1: Check if Autovacuum Is Enabled

```
SHOW autovacuum;
-- Must return 'on'. If 'off', this is the root cause.
```

Step 2: Identify Tables with Dead Tuple Accumulation

```
SELECT schemaname, relname, n_live_tup, n_dead_tup,
       round(n_dead_tup::numeric / NULLIF(n_live_tup + n_dead_tup, 0) * 10
0, 2)
         AS dead_pct,
       last_vacuum, last_autovacuum
FROM pg_stat_user_tables
WHERE n_dead_tup > 10000
ORDER BY n_dead_tup DESC
LIMIT 20;
```

Step 3: Estimate Table Bloat Using pgstattuple

```
CREATE EXTENSION IF NOT EXISTS pgstattuple;

SELECT * FROM pgstattuple('your_table_name');
-- Look at 'dead_tuple_percent'. Healthy is < 10%.
-- Values > 20% indicate significant bloat.
```

Step 4: Check for Transaction ID Wraparound Risk

```
SELECT datname,
       age(datfrozenxid) AS xid_age,
       2147483647 - age(datfrozenxid) AS remaining_before_wraparound
FROM pg_database
ORDER BY xid_age DESC;
```

If **xid_age** exceeds 200 million, this is a warning. Beyond 1 billion, it becomes urgent. At approximately 2 billion, PostgreSQL enters a protective shutdown.

Step 5: Check Autovacuum Worker Activity

```
SELECT pid, datname, relid::regclass, phase,
       heap_blks_total, heap_blks_scanned, heap_blks_vacuumed
FROM pg_stat_progress_vacuum;
```

2.4 Resolution & Remediation

Immediate Actions

1. Run a manual VACUUM on the most bloated tables:

```
VACUUM (VERBOSE) your_bloated_table;
```

2. For severe bloat, use VACUUM FULL (requires exclusive lock and downtime):

```
-- WARNING: VACUUM FULL rewrites the entire table. Plan a maintenance wind
ow.
VACUUM FULL VERBOSE your_bloated_table;
```

3. Alternatively, use pg_repack for online space reclamation without exclusive locks:

```
-- Install pg_repack extension first
pg_repack -d your_database -t your_bloated_table --no-superuser-check
```

Long-Term Autovacuum Tuning

Adjust autovacuum settings at the table level for high-churn tables:

```
ALTER TABLE your_table SET (autovacuum_vacuum_threshold = 50);
ALTER TABLE your_table SET (autovacuum_vacuum_scale_factor = 0.02);
ALTER TABLE your_table SET (autovacuum_analyze_threshold = 50);
ALTER TABLE your_table SET (autovacuum_analyze_scale_factor = 0.01);
```

Adjust global settings in **postgresql.conf** for busy systems:

Parameter	Default	Recommended	Purpose
autovacuum_max_workers	3	5–8	More parallel vacuum workers
autovacuum_naptime	1min	15–30s	Frequency of autovacuum checks
autovacuum_vacuum_cost_delay	2ms	0–2ms	Throttling delay per cost unit
autovacuum_vacuum_cost_limit	200	400–1000	Work done per cycle before pausing
maintenance_work_mem	64MB	512MB–1GB	Memory for VACUUM operations

> **PostgreSQL 17 Note**
> In PostgreSQL 17, cost-based delays have been further optimized. VACUUM can now skip pages more efficiently using new visibility map improvements, significantly speeding up large table vacuuming.

2.5 Monitoring Recommendations

- Set up alerts when n_dead_tup exceeds a threshold (e.g., 100,000 or dead_pct > 20%).
- Monitor **age(datfrozenxid)** and alert when it exceeds 200 million.
- Track last_autovacuum timestamps — if a table has not been vacuumed in over 24 hours during active workloads, investigate.

- Use **pg_stat_activity** to find long-running transactions that may block VACUUM.

3. Issue 2: Missing or Inefficient Indexes & Poor Query Plans

3.1 What Is the Problem?

PostgreSQL relies heavily on indexes to locate rows efficiently. When appropriate indexes are missing, the query planner resorts to sequential scans — reading every row in the table. On large tables, this results in extremely slow queries, high CPU consumption, excessive disk I/O, and increased shared buffer pressure. Equally problematic are unused indexes that consume storage and slow down write operations.

Poor query plans can also arise from stale or inaccurate table statistics, which cause the query planner to make suboptimal decisions about join strategies, scan methods, and row count estimates.

3.2 Root Causes

- Tables that have grown significantly since initial schema design, with no corresponding index review.
- Application queries evolved over time (new WHERE clauses, JOIN conditions) without DBA-led index analysis.
- Stale statistics: ANALYZE has not been run recently or autovacuum ANALYZE thresholds are too high.
- Over-indexing: too many indexes on write-heavy tables, degrading INSERT/UPDATE performance.
- Missing composite indexes for multi-column WHERE clauses or covering indexes for index-only scans.
- Implicit type casts in queries preventing index usage (e.g., comparing varchar with integer).

3.3 Detection & Diagnosis

Step 1: Identify Slow Queries

```
-- Enable pg_stat_statements if not already
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

-- Top 20 queries by total execution time
SELECT queryid, calls, total_exec_time::numeric(12,2) AS total_ms,
       mean_exec_time::numeric(12,2) AS avg_ms,
       rows, query
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 20;
```

Step 2: Analyze Query Execution Plans

```
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT) SELECT ... ;

-- Look for:
-- * Seq Scan on large tables (missing index)
-- * Nested Loop with high row estimates (missing join index)
-- * Sort with high memory usage (missing ORDER BY index)
-- * Filter lines removing most rows (index pushdown needed)
-- * Actual rows >> Estimated rows (stale statistics)
```

Step 3: Find Missing Indexes (Sequential Scans on Large Tables)

```
SELECT schemaname, relname, seq_scan, seq_tup_read,
       idx_scan, n_live_tup,
       round(seq_tup_read::numeric / NULLIF(seq_scan, 0), 0)
         AS avg_rows_per_scan
FROM pg_stat_user_tables
WHERE seq_scan > 100 AND n_live_tup > 10000
ORDER BY seq_tup_read DESC LIMIT 20;
```

Tables with high **seq_scan** counts AND large **n_live_tup** values are prime candidates for indexing.

Step 4: Find Unused Indexes (Candidates for Removal)

```
SELECT schemaname, indexrelname, relname, idx_scan,
       pg_size_pretty(pg_relation_size(indexrelid)) AS idx_size
FROM pg_stat_user_indexes
WHERE idx_scan = 0
  AND indexrelid NOT IN (
    SELECT indexrelid FROM pg_index WHERE indisprimary OR indisunique)
ORDER BY pg_relation_size(indexrelid) DESC;
```

Step 5: Check Statistics Freshness

```
SELECT schemaname, relname, last_analyze, last_autoanalyze,
       n_mod_since_analyze
FROM pg_stat_user_tables
WHERE n_mod_since_analyze > 10000
ORDER BY n_mod_since_analyze DESC;
```

3.4 Resolution & Remediation

Immediate Actions

1. Run ANALYZE on tables with stale statistics:

```
ANALYZE VERBOSE your_table;
```

2. Create indexes for identified slow queries (use CONCURRENTLY to avoid locks):

```
-- Basic B-tree index
CREATE INDEX CONCURRENTLY idx_orders_customer_id ON orders (customer_id);

-- Composite index for multi-column WHERE
CREATE INDEX CONCURRENTLY idx_orders_status_date
  ON orders (status, created_at DESC);

-- Covering index for index-only scans
CREATE INDEX CONCURRENTLY idx_orders_covering
  ON orders (customer_id) INCLUDE (total_amount, status);
```

3. Remove confirmed unused indexes:

```
DROP INDEX CONCURRENTLY idx_unused_index_name;
```

Query Plan Optimization

When EXPLAIN shows inaccurate row estimates, increase the statistics target:

```
ALTER TABLE your_table ALTER COLUMN your_column SET STATISTICS 500;
ANALYZE your_table;
```

> **PostgreSQL 16/17 Enhancement**
> PostgreSQL 16 introduced incremental sort improvements and better join planning with Memoize nodes. PostgreSQL 17 added further planner improvements including better handling of OR-clause indexing and improved partition pruning. Always re-test execution plans after major version upgrades.

3.5 Monitoring Recommendations

- Track **pg_stat_statements** weekly — watch for regressions in mean_exec_time after deployments.
- Alert when seq_scan / idx_scan ratio exceeds a threshold on tables with > 100K rows.
- Periodically audit indexes: remove unused ones, consolidate overlapping ones.
- Use **auto_explain** extension with auto_explain.log_min_duration to capture slow query plans automatically.

4. Issue 3: Lock Contention & Long-Running Transactions

4.1 What Is the Problem?

PostgreSQL uses a sophisticated locking mechanism to maintain data integrity during concurrent operations. Lock contention occurs when multiple sessions compete for locks on the same resources — tables, rows, or advisory locks. When combined with long-running transactions, this creates a cascading effect: blocked sessions queue up, connection pools fill, and the application becomes unresponsive.

In replication setups, long-running transactions on the primary delay WAL segment recycling, and long queries on standbys with *hot_standby_feedback* enabled prevent the primary from vacuuming dead tuples.

4.2 Root Causes

- Application code that holds transactions open while waiting for user input or external API calls.
- Missing or misconfigured **statement_timeout** and **idle_in_transaction_session_timeout**.
- DDL operations (ALTER TABLE, CREATE INDEX without CONCURRENTLY) taking AccessExclusive locks.
- Deadlocks caused by inconsistent lock ordering across application paths.
- ORM-generated queries that inadvertently lock more rows than necessary (e.g., SELECT … FOR UPDATE on broad result sets).
- Batch jobs and reporting queries running during peak traffic hours.

4.3 Detection & Diagnosis

Step 1: Identify Current Locks and Blocked Sessions

```
SELECT blocked.pid AS blocked_pid,
       blocked_activity.usename AS blocked_user,
       blocked_activity.query AS blocked_query,
       blocking.pid AS blocking_pid,
       blocking_activity.usename AS blocking_user,
       blocking_activity.query AS blocking_query,
       blocking_activity.state AS blocking_state
FROM pg_locks blocked
JOIN pg_stat_activity blocked_activity
  ON blocked.pid = blocked_activity.pid
JOIN pg_locks blocking
  ON blocking.locktype = blocked.locktype
  AND blocking.relation = blocked.relation
  AND blocking.pid != blocked.pid
JOIN pg_stat_activity blocking_activity
  ON blocking.pid = blocking_activity.pid
WHERE NOT blocked.granted;
```

Step 2: Find Long-Running Transactions

```
SELECT pid, usename, state, backend_start, xact_start,
       now() - xact_start AS xact_duration,
       now() - query_start AS query_duration,
       wait_event_type, wait_event, query
FROM pg_stat_activity
WHERE state != 'idle'
  AND xact_start IS NOT NULL
ORDER BY xact_start ASC LIMIT 20;
```

Step 3: Detect Idle-in-Transaction Sessions

```
SELECT pid, usename, state,
       now() - state_change AS idle_duration, query
FROM pg_stat_activity
WHERE state = 'idle in transaction'
  AND now() - state_change > interval '5 minutes'
ORDER BY state_change ASC;
```

Step 4: Check for Deadlocks

```
-- Check PostgreSQL log for deadlock entries:
-- grep -i 'deadlock' /var/log/postgresql/postgresql-*.log

-- Monitor deadlock count:
SELECT datname, deadlocks FROM pg_stat_database
ORDER BY deadlocks DESC;
```

4.4 Resolution & Remediation

Immediate Actions

1. Terminate blocking sessions if safe:

```
-- Graceful cancel (sends cancel to the query):
SELECT pg_cancel_backend(<blocking_pid>);

-- Force terminate (kills the session):
SELECT pg_terminate_backend(<blocking_pid>);
```

2. Configure safety timeouts in postgresql.conf:

```
idle_in_transaction_session_timeout = '5min'
statement_timeout = '30s'    -- Adjust per workload
lock_timeout = '10s'         -- Fail fast if lock unavailable
```

Long-Term Solutions

- Enforce short transactions in application code — open transaction, do work, commit immediately.
- Always use **CREATE INDEX CONCURRENTLY** for production DDL operations.
- Schedule batch jobs and maintenance outside peak hours using **pg_cron** or OS-level scheduling.
- Use connection pooling (**PgBouncer**) in transaction mode to limit connection exhaustion impact.
- On standbys: set **max_standby_streaming_delay** appropriately (e.g., 30s) to balance query availability with replication health.

> **Replication Impact**
> Long-running transactions on the primary prevent WAL segments from being recycled when replication slots are in use. This directly contributes to Issue 5 (Replication Slot Bloat). Fixing lock contention and long transactions often resolves replication lag as a side effect.

5. Issue 4: Suboptimal Memory & Configuration Settings

5.1 What Is the Problem?

PostgreSQL ships with conservative default configuration values designed to work on minimal hardware. In production, these defaults are almost always inadequate. Misconfigured memory settings lead to excessive disk I/O (data not cached in shared buffers), slow sorts and hash joins (work_mem too low, causing spills to disk), and poor checkpoint performance (I/O spikes and WAL write amplification).

On private cloud VMs, there is the additional complexity of VM resource allocation — ensuring PostgreSQL memory settings are properly sized relative to the VM's available RAM, avoiding swap, and accounting for OS filesystem cache.

5.2 Root Causes

- Using PostgreSQL defaults without tuning for the VM's hardware profile.
- **shared_buffers** too small — the most impactful single setting, commonly left at the default 128MB.
- **work_mem** too low — causes sorts and hash operations to spill to temporary files on disk.
- **effective_cache_size** not reflecting actual available memory, leading to suboptimal planner choices.
- Checkpoint settings too aggressive (frequent checkpoints) or too relaxed (large I/O spikes).
- VM overcommitment: the hypervisor allocates more memory to VMs than physically available, causing swap.
- **huge_pages** not enabled on Linux, reducing TLB efficiency for large shared_buffers.

5.3 Detection & Diagnosis

Step 1: Check Buffer Cache Hit Ratio

```
SELECT
  sum(heap_blks_read) AS heap_read,
  sum(heap_blks_hit) AS heap_hit,
  round(sum(heap_blks_hit)::numeric /
    NULLIF(sum(heap_blks_hit) + sum(heap_blks_read), 0)
    * 100, 2) AS hit_ratio
FROM pg_statio_user_tables;

-- Target: > 99% for OLTP. < 95% means shared_buffers too small.
```

Step 2: Detect Temporary File Spills (work_mem Too Low)

```
SELECT datname, temp_files, temp_bytes,
       pg_size_pretty(temp_bytes) AS temp_size
FROM pg_stat_database
WHERE temp_bytes > 0
ORDER BY temp_bytes DESC;

-- Enable logging: log_temp_files = 0  (logs all temp file usage)
```

Step 3: Evaluate Checkpoint Performance

```
SELECT checkpoints_timed, checkpoints_req,
       buffers_checkpoint, buffers_clean, maxwritten_clean,
       checkpoint_write_time, checkpoint_sync_time
FROM pg_stat_bgwriter;

-- checkpoints_req >> checkpoints_timed: max_wal_size too small
-- maxwritten_clean > 0: bgwriter can't keep up
```

Step 4: Check Current Configuration

```
SELECT name, setting, unit, source, context
FROM pg_settings
WHERE name IN ('shared_buffers', 'work_mem', 'maintenance_work_mem',
  'effective_cache_size', 'max_wal_size',
  'checkpoint_completion_target', 'wal_buffers',
  'random_page_cost', 'effective_io_concurrency',
  'huge_pages', 'max_connections')
ORDER BY name;
```

Step 5: Check VM-Level Memory (from OS)

```
# Check total and available memory
free -h

# Check if swap is in use (should be minimal)
swapon --show
vmstat 1 5  # Look at si/so columns for swap in/out
```

5.4 Resolution & Remediation

The following table provides recommended settings based on available VM RAM. These are starting points — adjust based on your specific workload:

Parameter	Guideline	Example (64GB VM)
shared_buffers	25% of VM RAM	16GB
effective_cache_size	50–75% of VM RAM	48GB
work_mem	RAM / max_connections / 4	64MB
maintenance_work_mem	1–2GB (or 5% of RAM)	2GB
max_wal_size	2–8GB	4GB
min_wal_size	1–2GB	1GB
checkpoint_completion_target	0.9	0.9
wal_buffers	64MB (if shared_buffers > 4GB)	64MB
random_page_cost	1.1–1.5 for SSD storage	1.1
effective_io_concurrency	200 for SSD storage	200
huge_pages	try	try

Enable Huge Pages on Linux

```
# Calculate required huge pages
# shared_buffers = 16GB = 16384MB, huge page = 2MB
# Required pages = 16384 / 2 + buffer = ~8500

# Set in /etc/sysctl.conf:
vm.nr_hugepages = 8500

# Apply: sysctl -p

# In postgresql.conf:
huge_pages = try
```

PostgreSQL 15/16/17 Note

PostgreSQL 15 improved WAL insertion locking. PostgreSQL 16 introduced I/O statistics views (pg_stat_io) for much better I/O analysis. PostgreSQL 17 adds further improvements to buffer management. Use pg_stat_io (v16+) to identify specific I/O bottlenecks by backend type and operation.

6. Issue 5: Streaming Replication Lag & Replication Slot Bloat

6.1 What Is the Problem?

In dual-node setups using streaming replication with replication slots, two closely related problems are common. **First, replication lag:** the standby falls behind the primary in applying WAL records. This means the standby has stale data, and in a failover scenario, recent transactions may be lost. **Second, replication slot bloat:** PostgreSQL retains WAL segments on the primary for as long as a slot consumer needs them. If the standby is down, disconnected, or severely lagged, WAL files accumulate, eventually filling the disk and causing a catastrophic primary failure.

These issues are especially dangerous because they can cascade: a disk-full primary due to WAL accumulation crashes, and the standby is too far behind to serve as a reliable failover target.

6.2 Root Causes

- Standby server is overloaded, has insufficient I/O bandwidth, or has less CPU/RAM than the primary.
- Network latency or bandwidth constraints between primary and standby VMs.
- Replication slot remains active after the standby has been decommissioned or rebuilt (**orphaned slot**).
- Long-running queries on the standby with *hot_standby_feedback = on*, preventing WAL advancement.
- Sudden burst of write activity on the primary that the standby cannot replay fast enough.
- **max_wal_senders** set too low, causing connection rejections.
- **wal_keep_size** too large or slot with no consumer retaining WAL segments indefinitely.

6.3 Detection & Diagnosis

Step 1: Check Replication Status (On Primary)

```
SELECT client_addr, state, sent_lsn, write_lsn,
       flush_lsn, replay_lsn,
       pg_wal_lsn_diff(sent_lsn, replay_lsn) AS replay_lag_bytes,
       pg_size_pretty(
         pg_wal_lsn_diff(sent_lsn, replay_lsn)
       ) AS replay_lag_pretty,
       write_lag, flush_lag, replay_lag
FROM pg_stat_replication;
```

Step 2: Check Replication Slots (Critical!)

```
SELECT slot_name, slot_type, active, active_pid,
       restart_lsn, confirmed_flush_lsn,
       pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)
         AS retained_bytes,
       pg_size_pretty(
         pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)
       ) AS retained_wal_size
FROM pg_replication_slots
ORDER BY
  pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn) DESC;
```

CRITICAL: If **active = false** and **retained_wal_size** is large (GBs), this slot is bloating your pg_wal directory.

Step 3: Check WAL Directory Size

```
-- From psql (v15+):
SELECT pg_size_pretty(sum(size)) AS total_wal_size
FROM pg_ls_waldir();

-- Or from bash:
du -sh /var/lib/postgresql/<version>/main/pg_wal/
```

Step 4: Check Disk Space on Primary

```
# Monitor the data directory filesystem
df -h /var/lib/postgresql/

# Alert threshold: < 20% free = warning, < 10% = critical
```

Step 5: Check Standby Replay Status (On Standby)

```
SELECT pg_is_in_recovery(),
       pg_last_wal_receive_lsn(),
       pg_last_wal_replay_lsn(),
       pg_last_xact_replay_timestamp(),
       now() - pg_last_xact_replay_timestamp() AS replay_delay;
```

6.4 Resolution & Remediation

Immediate Actions for Slot Bloat

1. Drop orphaned/inactive replication slots immediately:

```
-- Verify the slot is truly unused:
SELECT slot_name, active, active_pid FROM pg_replication_slots;

-- Drop the orphaned slot:
SELECT pg_drop_replication_slot('orphaned_slot_name');
```

2. Set maximum WAL retention per slot (PostgreSQL 13+):

```
-- In postgresql.conf:
max_slot_wal_keep_size = '10GB'
-- Slots invalidated if they fall behind more than this amount
```

This is a **critical safety net**. Without it, a single inactive slot can consume the entire disk.

Addressing Replication Lag

1. Ensure the standby has equivalent or better I/O and CPU resources than the primary.
2. Tune standby replay performance:

```
-- On standby, in postgresql.conf:
wal_receiver_timeout = '60s'
wal_receiver_status_interval = '10s'
```

3. On the primary, if write bursts are common:

```
-- Increase WAL sender resources:
max_wal_senders = 10
wal_sender_timeout = '60s'
```

Monitoring & Alerting Thresholds

Metric	Warning	Critical	Query / Command
Replication lag (seconds)	> 30s	> 120s	pg_stat_replication.replay_lag
Replication lag (bytes)	> 256MB	> 1GB	pg_wal_lsn_diff(sent, replay)
Inactive slot retained WAL	> 2GB	> 5GB	pg_replication_slots + lsn_diff
pg_wal directory size	> 10GB	> 20GB	pg_ls_waldir() or du -sh
Disk free space	< 20%	< 10%	df -h on data directory

PostgreSQL 15+ Slot Safety

PostgreSQL 15 introduced slot invalidation logging. PostgreSQL 16 added pg_stat_subscription_stats for better monitoring. PostgreSQL 17 improves slot synchronization for more reliable failover. Always set max_slot_wal_keep_size as a safety guardrail on all production systems.

7. Appendix: Quick Reference & Diagnostic Toolkit

7.1 Essential Diagnostic Extensions

Extension	Purpose	Install Command
pg_stat_statements	Track query execution statistics	CREATE EXTENSION pg_stat_statements;
pgstattuple	Table-level bloat analysis	CREATE EXTENSION pgstattuple;
pg_buffercache	Shared buffer content inspection	CREATE EXTENSION pg_buffercache;
auto_explain	Log slow query plans automatically	Loaded via shared_preload_libraries
pg_repack	Online table/index reorganization	Install OS package + CREATE EXTENSION

7.2 Daily Health Check Script

Run this composite query set as part of a daily DBA health check:

```
-- 1. Database sizes
SELECT datname, pg_size_pretty(pg_database_size(datname))
FROM pg_database;

-- 2. Top bloated tables
SELECT relname, n_dead_tup, last_autovacuum
FROM pg_stat_user_tables
WHERE n_dead_tup > 10000 ORDER BY n_dead_tup DESC LIMIT 10;

-- 3. Cache hit ratio
SELECT round(sum(heap_blks_hit)::numeric /
  NULLIF(sum(heap_blks_hit) + sum(heap_blks_read), 0)
  * 100, 2) AS cache_hit FROM pg_statio_user_tables;

-- 4. Replication status
SELECT client_addr, state,
  pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS lag
FROM pg_stat_replication;

-- 5. Slot health
SELECT slot_name, active,
  pg_size_pretty(pg_wal_lsn_diff(
    pg_current_wal_lsn(), restart_lsn)) AS retained
FROM pg_replication_slots;

-- 6. Long-running transactions
SELECT pid, usename, now() - xact_start AS duration, query
FROM pg_stat_activity
WHERE state != 'idle'
  AND xact_start < now() - interval '5m';

-- 7. XID wraparound age
SELECT datname, age(datfrozenxid) AS xid_age
FROM pg_database ORDER BY 2 DESC;
```

7.3 Recommended Monitoring Stack

- **Prometheus + postgres_exporter:** Collect all key metrics from pg_stat_* views.
- **Grafana:** Visualize performance dashboards with alerting.
- **pgBadger:** Analyze PostgreSQL log files for slow queries, lock waits, and errors.
- **pg_stat_monitor (Percona):** Enhanced alternative to pg_stat_statements with histograms and query plan capture.
- **check_postgres (Nagios/Icinga):** Pre-built checks for replication lag, bloat, connections, and more.